Oh InSeong(오인성)

Android Developer

Email superohinsung@naver.com

GitHub github.com/ois0886

Blog superohinsung.tistory.com

PROJECTS

프로젝트 목차

Project. 1 내눈 키오스크
스마트 헬스케어 키오스크 서비스 Android 앱 — 03p ~ 08p

Project. 2 모리, Mo-Re
서버 상태와 성능을 시각화하는 Android 모니터링 앱 — 09p ~ 13p

Project. 3 Glim
글귀를 공유하며 숏츠 폼으로 감성을 공유하는 Android 앱 — 14p ~ 19p

PROJECTS. 1

내눈 키오스크

2025.10.10 ~ 2025.11.20 · 6명(IoT 1명, Android 5명) · Android 키오스크 코드 개선 및 리팩토링

간단 소개

생체정보 통합 플랫폼을 통해 고령층의 건강 측정과 관리, 광고 연계를 지원하는 스마트 헬스케어 키오스크 서비스

배경

(주)픽셀로의 서비스 **내눈 키오스크**를 운영 환경에 맞게 리팩토링 및 요구사항 개선. SSAFY 13기 기업연계 프로젝트로 진행했으며, 코드 반출이 허용되지 않아 구현 구조와 문제 해결 과정 중심으로 정리.

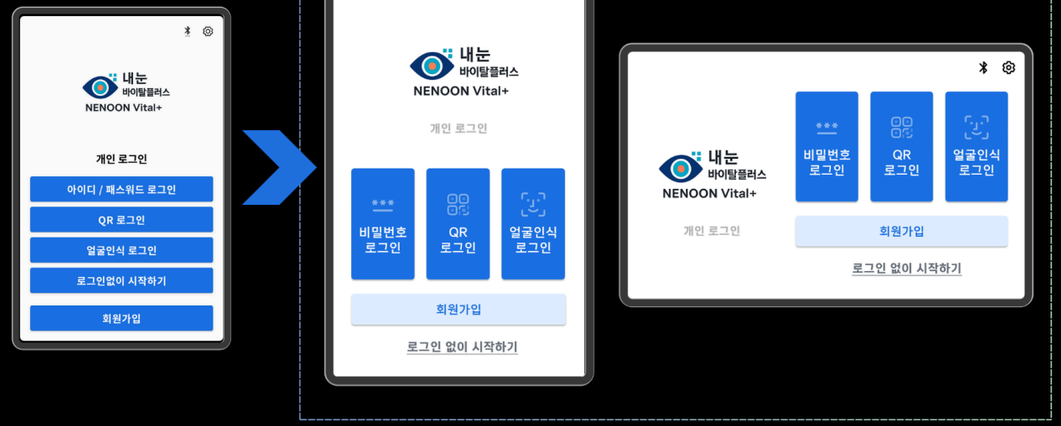
핵심 기여

- 부분적 XML UI를 **Jetpack Compose UI 100%**로 전환
- Compose State Hoisting 패턴 적용으로 UI 컴포넌트 재사용성 향상 및 코드 중복 제거
- 레거시 패키지 구조를 기능·계층 기준으로 재구성해 의존성 방향과 유지보수성 개선

기술 스택

Kotlin | **Android Jetpack** | **Jetpack Compose** | **ViewModel** | **Coroutine** | **Flow**

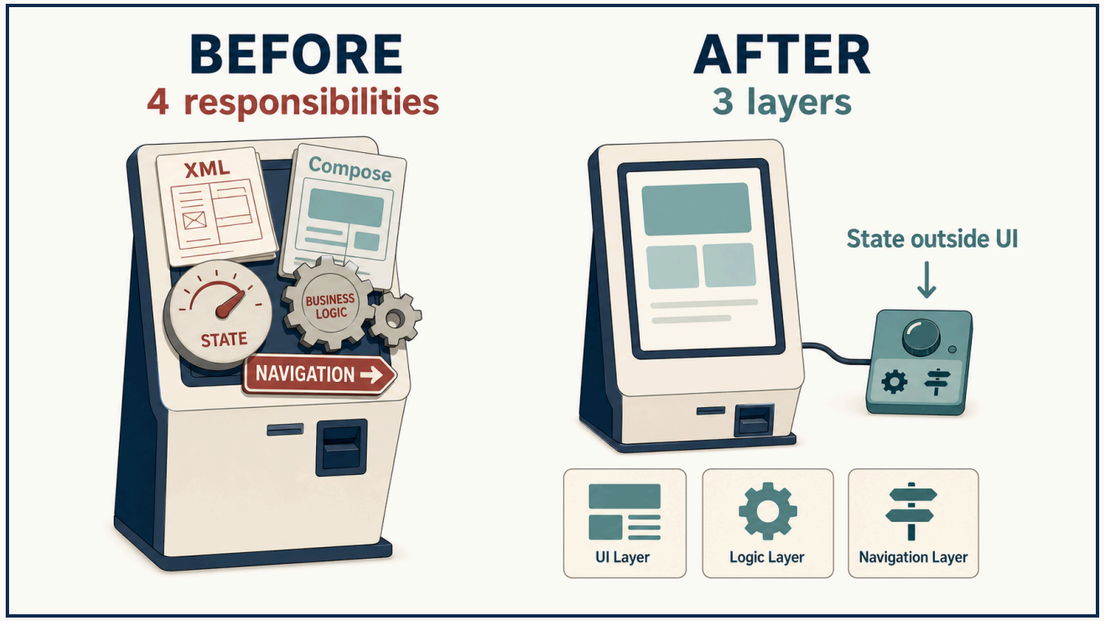
UX/UI 개선 – 로그인 화면



UX/UI 개선 – 검사 결과 화면

PROBLEM SOLVING 1

XML·Compose 혼용 UI 개선 / State Hoisting 구조화



ChatGPT로 제작한 이미지입니다.

문제

- 데이터 로딩, 상태 관리, 텍스트 구성, 버튼 클릭 처리가 하나의 화면 컴포넌트에 뒤섞여 유지보수 비용 증가
- XML View와 Compose 코드, ViewModel·비즈니스 로직·네비게이션 로직이 섞여 공통 정책 변경 시 중복 수정 발생

원인

화면별로 XML, Compose, ViewModel, 네비게이션 책임이 섞여 공통 UI 정책과 상태 처리 기준이 통일되지 않음.

해결

- Compose State Hoisting을 도입해 상태 생성·비즈니스 로직·네비게이션 의사결정을 Route/ViewModel 계층으로 이동
- 타이틀, 설명, 표/리스트, 버튼, 체크 영역을 재사용 가능한 컴포넌트로 분리
- 기능 단위 패키지 구조와 Route/Screen/Component 구조로 확장 가능한 화면 구조 정리

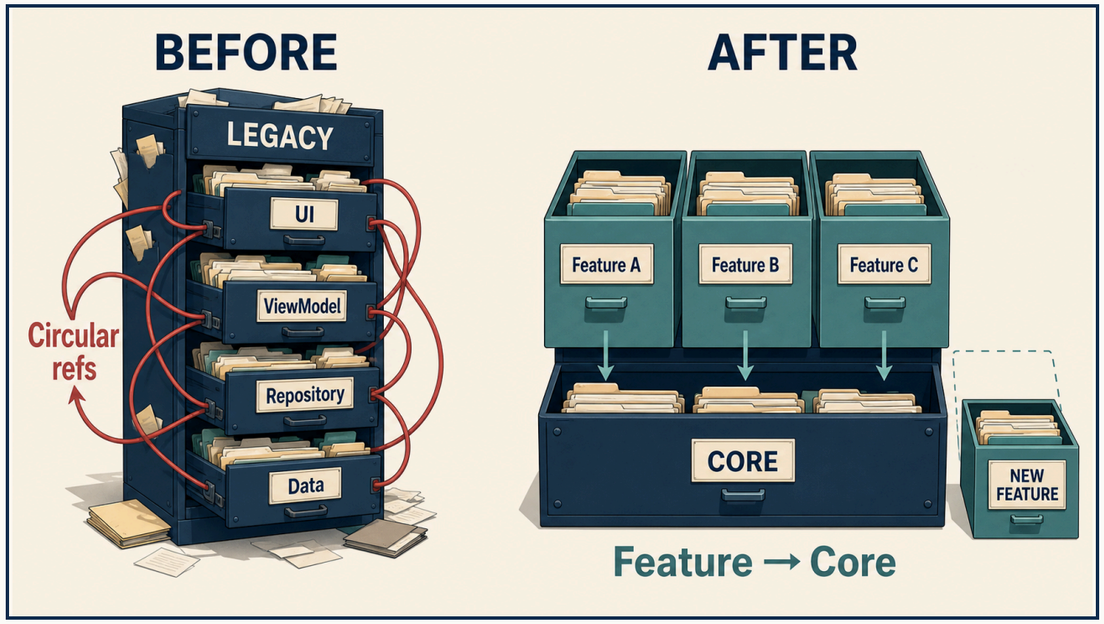
결과

- **Route → Screen → Component** 계층 구조로 텍스트 교체, 레이아웃 변경, 버튼 추가 시 수정 범위 감소
- 다국어, 폰트 크기, 버튼 스타일 같은 공통 UI 정책을 한 번에 반영할 수 있는 구조 확보

대체안 / 회고

ViewModel 책임 확장이나 XML 유지 + 부분 Compose 도입도 가능했지만, 선언적 UI·프리뷰·State Hoisting 장점을 살리기 위해 Screen/Component 중심 구조를 선택. XML과 Compose 혼용은 장기적으로 일관성과 생산성을 떨어뜨릴 수 있음을 확인.

레거시 패키지 구조 재편



ChatGPT로 제작한 이미지입니다.

문제

패키지 구조가 뒤섞이고 의존성이 순환 참조되어 코드 파악이 어렵고, 신규 기능 추가 시 사이드 이펙트가 예측 불가능해 개발 진행이 어려웠음.

원인

단일 패키지에 모든 화면·로직·데이터가 몰려 있고, ViewModel·Repository·UI가 서로 참조하며 의존성 방향이 불명확했음.

해결

- 기능 단위(feature)로 패키지를 재편
- 각 feature를 Route, Screen, Component, ViewModel로 계층화해 의존성 방향 정리
- 공통 로직은 core 영역으로 분리해 중복과 순환 참조 가능성 축소

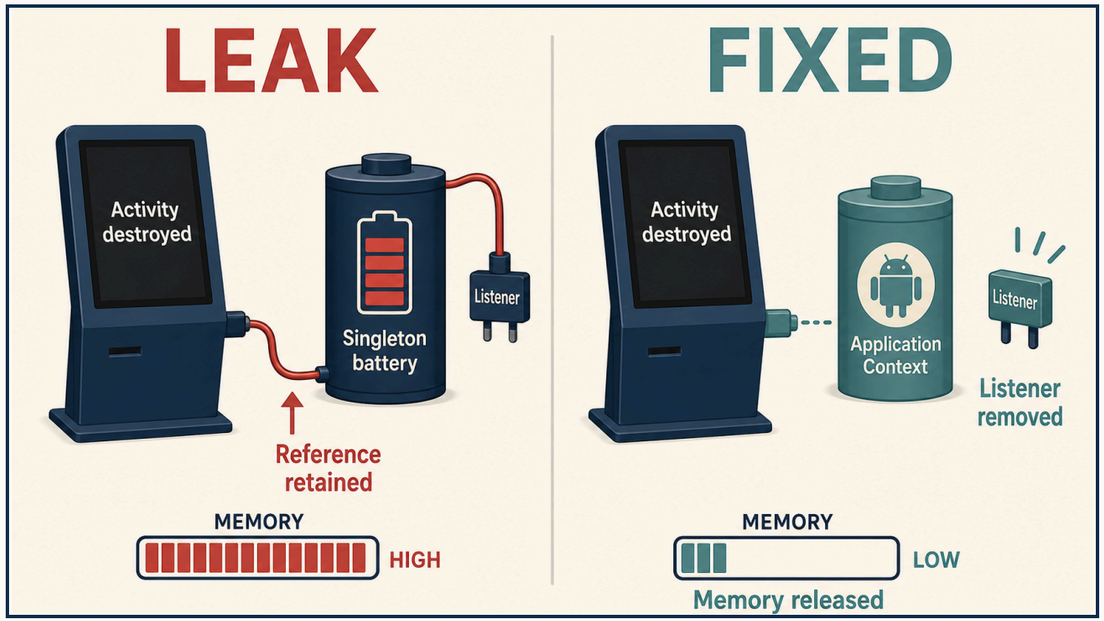
결과

- 기존 레거시 패키지 구조를 기능·계층 기준으로 재구성해 유지보수성 개선
- 신규 화면 추가 시 필요한 수정 위치가 명확해지고 사이드 이펙트 추적이 쉬워짐

대체안 / 회고

기존 패키지 유지 + 공통 유틸 분리도 가능했으나, 구조 자체가 바뀌지 않으면 의존성 문제가 재발할 것이라 feature 단위 재설계를 선택.

메모리 누수 개선



ChatGPT로 제작한 이미지입니다.

문제

Singleton 남용, Context 참조 오류, Listener 미해제 등으로 장시간 실행 환경에서 메모리 누수와 성능 저하 위험이 존재함.

원인

- Singleton에서 Activity Context를 직접 참조하거나 생명주기가 긴 객체가 짧은 생명주기 Context를 보유
- Callback·Listener를 등록 후 해제하지 않아 참조가 유지됨

해결

- Activity Context를 Application Context로 전환하고 Hilt 의존성 주입으로 Singleton 사용 최소화
- onDispose·onCleared에서 Listener와 리소스를 명시적으로 해제
- Compose에서 remember + DisposableEffect로 생명주기 관리
- 순환 참조 구조 제거 및 WeakReference 활용

결과

- 생명주기보다 오래 유지되던 Context·Listener 참조를 줄여 메모리 누수 가능성 개선
- 패키지 구조 정리와 의존성 개선이 장시간 실행 환경의 안정성 개선으로 이어짐

회고

구조적 문제 해결이 성능 개선으로 이어질 수 있음을 경험. 메모리 누수는 단일 코드 문제가 아니라 참조 방향과 생명주기 설계 문제로 바라봐야 함을 확인.

RESULT

내눈 키오스크 성과 및 회고

성과 및 결과

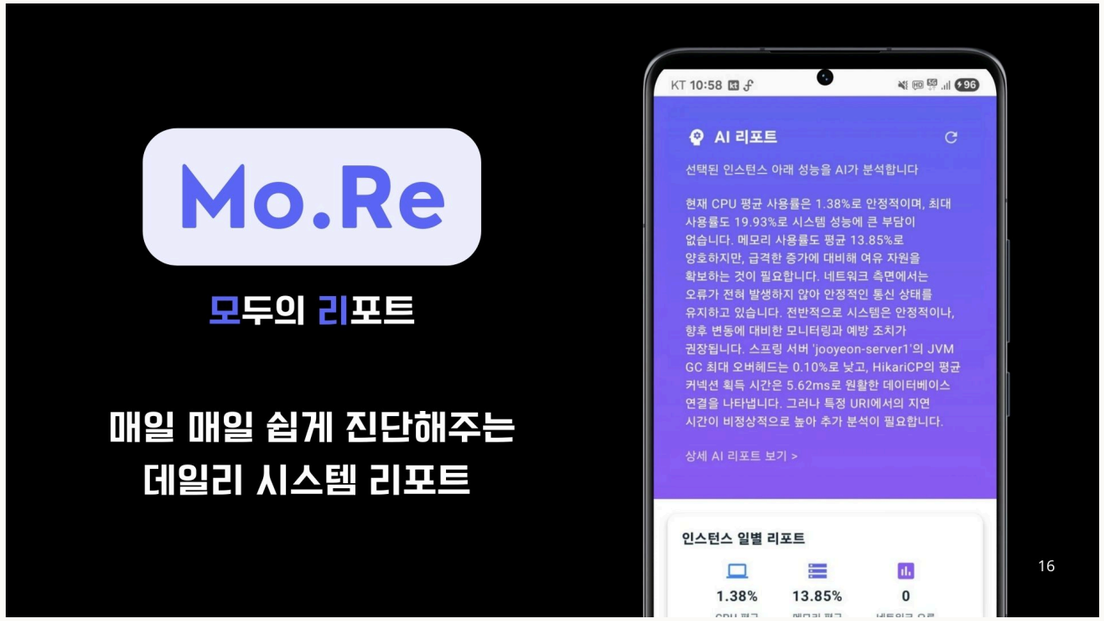
- SSAFY 13기 구미캠퍼스 자율 프로젝트 우수상 수상
- **상용 키오스크 앱**의 XML·Compose 혼용 UI를 Compose 중심 구조로 정리하고 Route-Screen-Component 책임 분리
- Activity Context 참조, Listener 미해제, Singleton 참조 구조를 개선해 장시간 실행 환경의 메모리 누수 위험 요소 제거
- 기능·계층 기준 패키지 재편과 공통 UI 컴포넌트 분리로 수정 영향 범위와 중복 수정 비용 감소

회고

- 처음에는 새로운 기능 개발을 중심으로 생각했지만, 기업이 실제로 필요로 한 것은 기존 앱을 운영 환경에 맞게 안정화하는 일이었습니다.
- 기대했던 방향과 다른 과제였지만, 팀원들과 요구사항의 의미를 다시 맞추며 "지금 사용자와 기업에 필요한 개발"에 집중하는 태도를 배웠습니다.
- 기존 코드와 요구사항을 충분히 이해하지 못한 상태에서 설계를 시작하면 수정이 반복된다는 것을 겪었고, 이후에는 레거시 인벤토리화, 사전 설계, 수정 범위 파악을 먼저 수행하는 것이 리팩토링의 핵심이라고 보게 되었습니다.
- 한 화면에 데이터 로딩, 상태 처리, 클릭 이벤트, 네비게이션이 몰려 있던 구조를 보며, 리팩토링은 단순한 UI 변경이 아니라 상태 소유 지점과 렌더링 책임을 나누는 운영 관점의 재설계라는 점을 체감했습니다.

모리, Mo-Re

2025.08.25 ~ 2025.09.29 · 6명(Backend 4명, Android 2명) · Android 개발(기여도 80%)



간단 소개

누구나 쉽게 이해할 수 있는 서버 리포트를 제공하여 서비스 상태와 성능을 시각화하고 관리·운영 효율성을 높이는 앱 서비스

배경

모바일 환경에서도 손쉽게 서버 상태를 확인할 수 있는 모니터링 도구의 필요성에 따라 프로젝트 기획. 코드 반출 불가로 구현 구조와 문제 해결 과정 중심으로 정리.

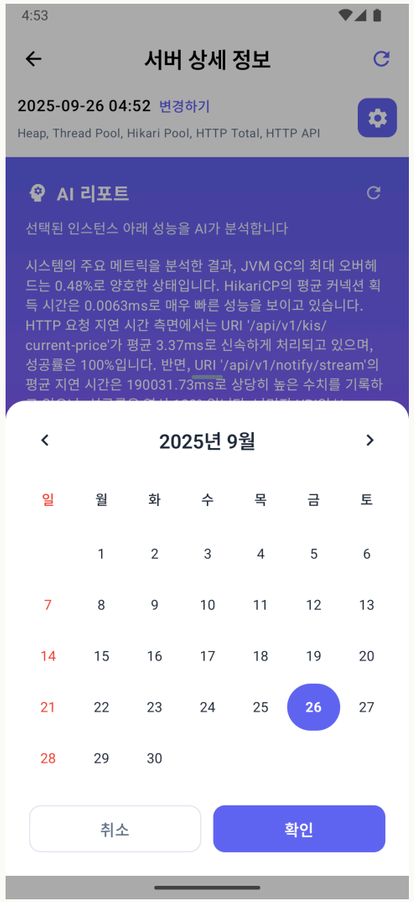
핵심 기여

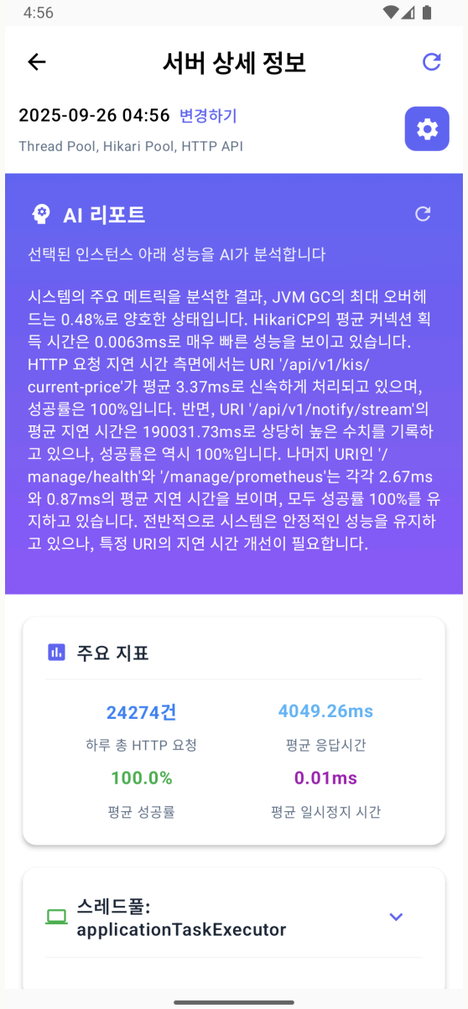
- 멀티 모듈 아키텍처 설계 및 Convention Plugin 기반 Build-logic 구축
- Custom Compose UI로 시계열 차트(LineChart) 구현
- Kotlin Coroutine async/await를 활용한 6개 API 비동기 처리로 **데이터 로딩 시간 56% 단축**

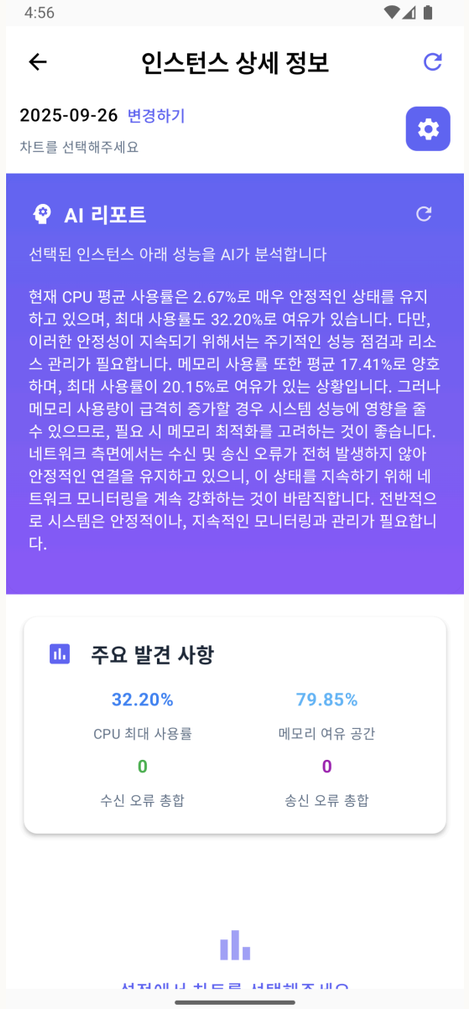
기술 스택

Kotlin **Jetpack Compose** **Orbit MVI** **Hilt** **Retrofit2** **OkHttp3** **Navigation3** **DataStore** **Coroutines**

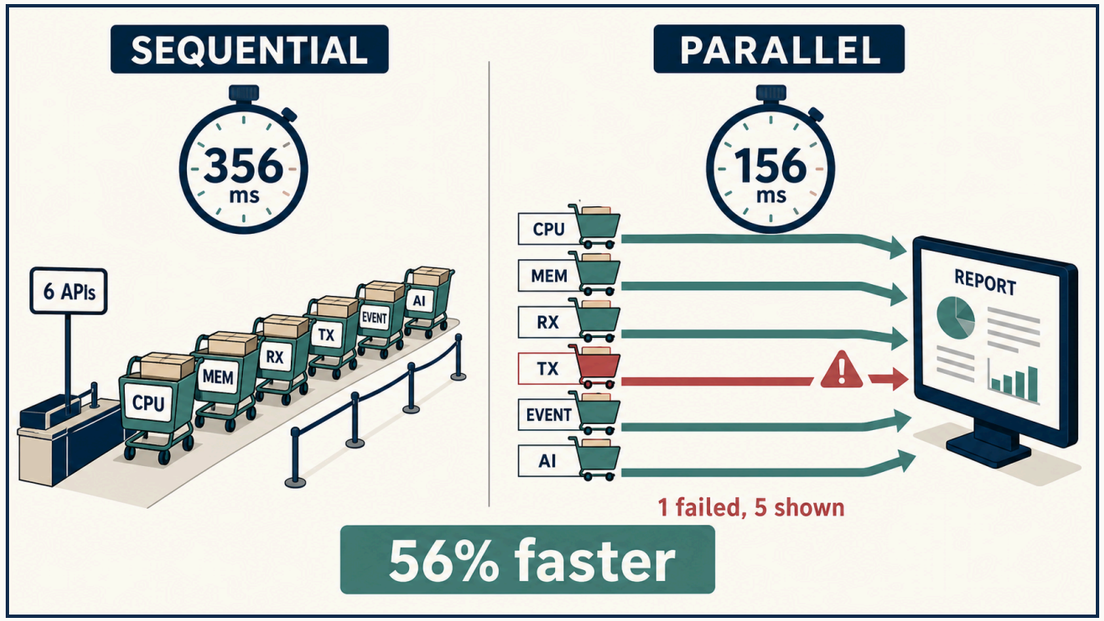
APP SCREENSHOT







6개 API 병렬 처리와 부분 실패 대응



ChatGPT로 제작한 이미지입니다.

문제

- CPU, 메모리, 네트워크 수신/송신, 이벤트 요약, AI 리포트 6개 API를 순차 호출해 약 356ms 로딩 발생
- 한 메트릭 API 실패가 전체 리포트 실패로 이어질 수 있고, 어떤 API가 실패했는지 추적하기 어려움

원인

API를 하나씩 await하는 직렬 구조로 네트워크 지연이 누적되고, 실패 전파 범위와 부분 데이터 표시 기준이 명확하지 않았음.

해결

- supervisorScope와 async를 사용해 6개의 메트릭 API를 독립 코루틴으로 병렬 실행
- 한 API 실패가 전체 코루틴 취소로 전파되지 않도록 각 비동기 블록 내부에서 runCatching 처리
- 실패한 메트릭 종류와 원인을 개별 로그로 기록하고, 성공한 메트릭은 부분 데이터로 화면에 반영

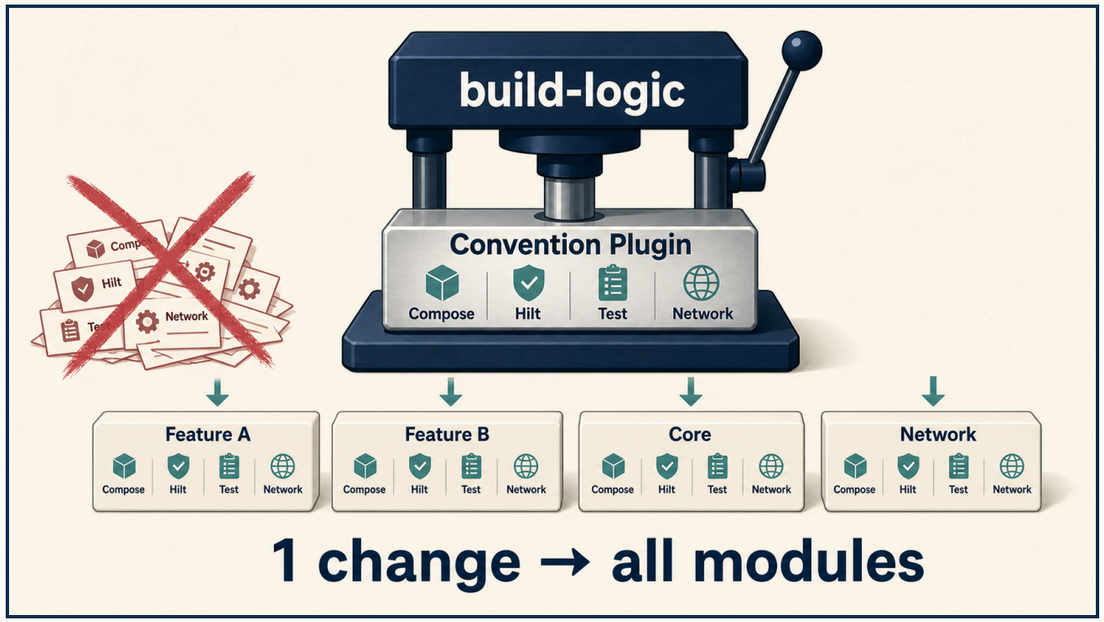
결과

- 메트릭 API 병렬 호출 구조로 **전체 로딩 시간 약 56% 단축**
- 특정 메트릭만 실패해도 나머지 데이터로 리포트 화면을 유지하는 장애 허용 UI 구조 확보
- 실패 API가 명확히 남아 디버깅과 장애 원인 추적이 쉬워짐

대체안 / 회고

서버 단일 집계 응답도 가능했지만, 부분 실패 처리와 클라이언트 전용 UI 계산 유연성에 제약이 있어 클라이언트 병렬 처리를 선택. API 병렬 처리는 속도뿐 아니라 부분 실패 상황에서도 UX를 유지하는 구조를 만드는 작업임을 확인.

Convention Plugin 기반 멀티 모듈 빌드 표준화



ChatGPT로 제작한 이미지입니다.

문제

멀티 모듈 프로젝트에서 Compose, Hilt, Test, Network 설정이 모듈마다 흩어져 중복되고 일관성 유지가 어려움.

원인

공통 빌드 설정을 각 모듈에서 수동 작성하면서 신규 모듈 생성 시 설정 누락, 버전 불일치, 옵션 차이가 반복될 수 있는 구조였음.

해결

- build-logic 모듈에 Convention Plugin을 정의해 공통 빌드 설정과 의존성을 중앙 관리
- Feature, Core, Network, Datastore, Hilt, Compose, Test, Quality 등 기능별 Gradle 플러그인을 분리 설계
- 각 기능 모듈에서는 id("mori.android.feature") 같은 단일 플러그인 적용만으로 일관된 설정 자동 적용

결과

- 빌드 설정 수정 시 build-logic 한 곳만 변경하면 전체 모듈에 반영
- 새 Feature/Core 모듈 추가 시 Convention Plugin 한 줄 적용으로 네이밍 규칙, Compose/Hilt 설정, 공통 의존성 자동 구성
- 모듈 생성 비용과 Gradle 설정 오류 가능성을 줄이고 멀티 모듈 확장 기준을 표준화

대체안 / 회고

루트 build.gradle.kts, apply(from), Version Catalog만 쓰는 방식도 고려했지만 플러그인·설정·의존성 묶음 적용에는 Convention Plugin이 더 적합하다고 판단. 초기 비용은 있지만 버전 정합성과 확장성을 중앙에서 제어할 수 있어 유지보수성이 향상됨.

모리, Mo-Re 성과 및 회고

성과 및 결과

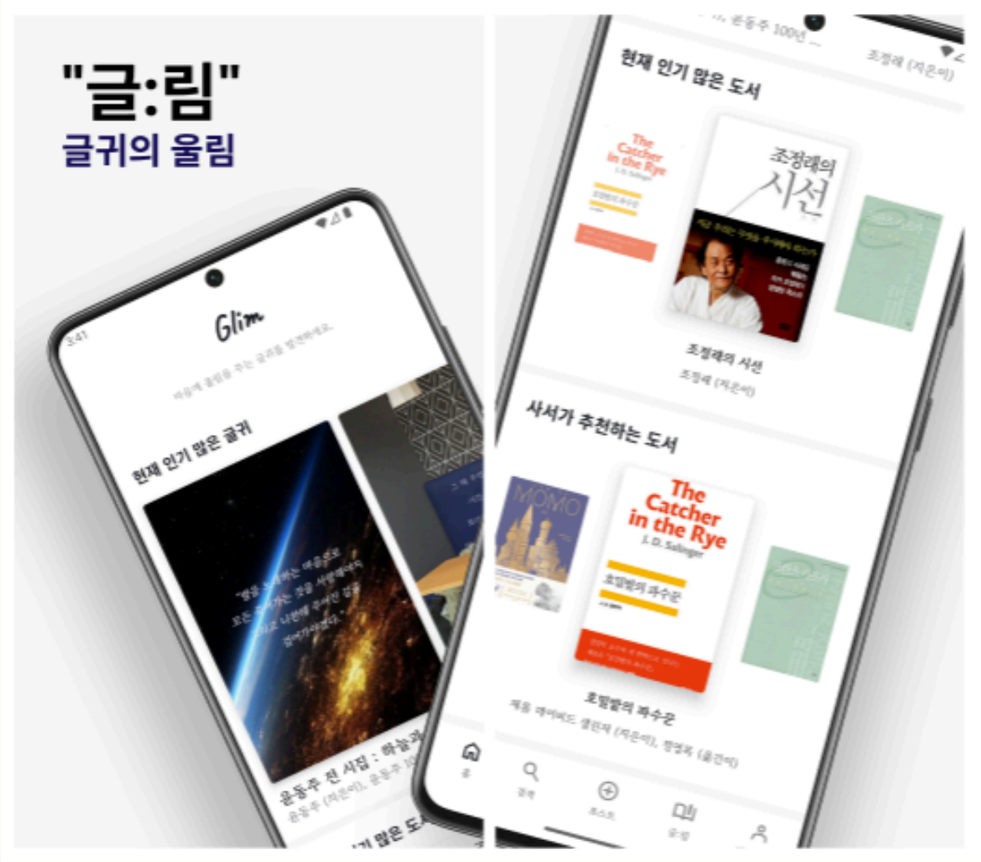
- 멀티 모듈 아키텍처 도입으로 기능별 모듈 분리 및 재사용성 향상
- async/await를 활용한 API 병렬 처리로 **데이터 로딩 시간 56% 개선**, 약 156ms로 단축
- **10개 이상의 Feature 모듈**과 재사용 가능한 Design System 컴포넌트 구현
- Convention Plugin 기반 build-logic 구축으로 모듈별 Gradle 설정 중복 제거 및 빌드 설정 일관성 확보

회고

- 프로젝트 중 팀원들이 예비군이나 개인 일정으로 자리를 비우는 상황이 반복되면서 계획한 속도대로만 진행하기 어려웠습니다.
- 그때마다 남은 인원이 할 수 있는 일을 다시 나누고, 막힌 부분과 다음 목표를 짧게 공유하며 프로젝트 흐름을 유지하려고 했습니다.
- 이 과정에서 멀티 모듈 구조와 build-logic은 단순한 기술 선택이 아니라, 인원이 바뀌거나 기능이 늘어도 각자 맡은 영역을 예측 가능하게 만드는 협업 장치라는 점을 체감했습니다.
- 좋은 구조는 코드 안에만 있는 것이 아니라, 팀이 불확실한 상황에서도 같은 방향을 바라보게 만드는 작업 방식과 함께 작동해야 한다는 점을 배웠습니다.

Glim

2025.07.07 ~ 2025.08.18 · 6명(Backend 2명, Android 2명, Frontend 1명, Backend & Android 1명) · Android 앱 개발



간단 소개

글귀를 공유하며, 숏츠(글림으로 명명) 폼으로 다른 사람들과 공유하는 앱

배경

무분별한 숏폼 콘텐츠 소비로 인한 집중력 저하 및 독서량 감소에 대응하여, 글귀 기반 디지털 디톡스 접근을 제시.

핵심 기여

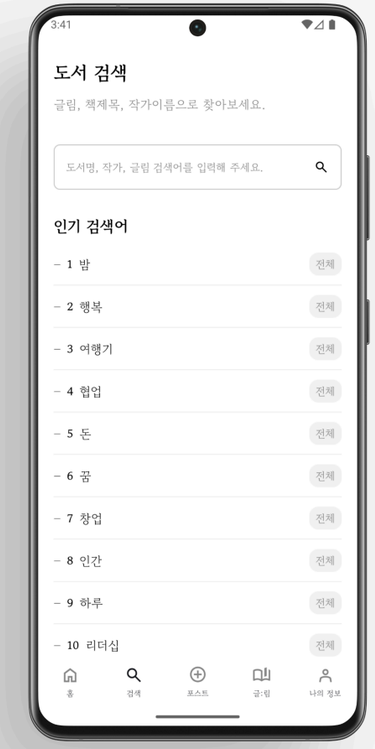
- Orbit MVI 패턴 적용을 통한 예측 가능한 상태 관리 및 테스트 용이성 향상 아키텍처 설계
- Kotlin Coroutine과 DataStore를 활용한 Session, Refresh Token 관리 시스템 구현
- Channel 기반 Navigator 패턴 도입으로 Screen과 Navigation 로직 완전 분리
- JUnit, MockK 기반 ViewModelTest를 **130+ 케이스** 구현

기술 스택

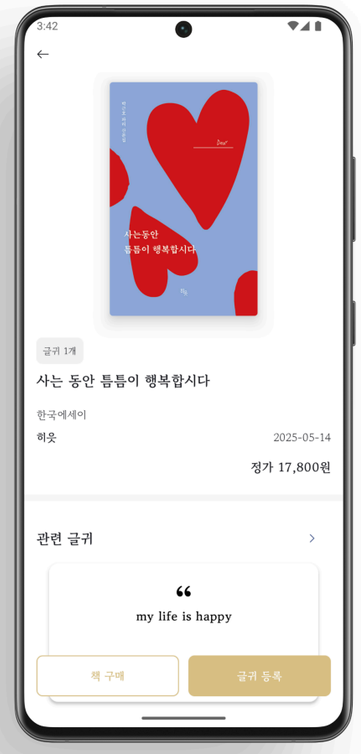
Kotlin **Jetpack Compose** **Hilt** **Orbit MVI** **Coroutine** **Retrofit3** **DataStore** **MockK** **Turbine** **JUnit**

AI로 이미지 제작

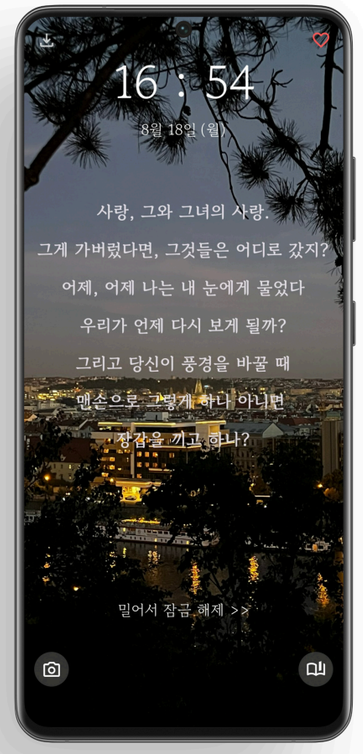
책과 글귀의 손쉬운 탐색



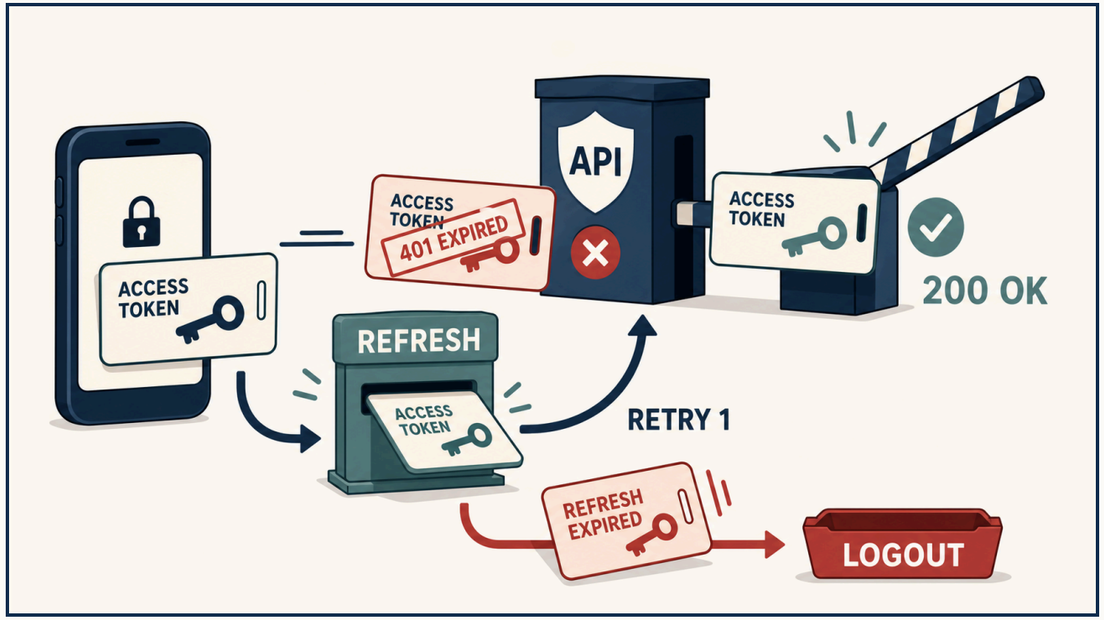
책의 쉬운 접근



잠금화면에서 글림 확인



Token 재발급과 인증 예외 처리 구조화



ChatGPT로 제작한 이미지입니다.

문제

Access Token 만료, Refresh Token 만료, 401 응답 재시도, 최대 재시도 초과, 로그아웃 사유 전달을 일관되게 처리하기 어려움.

원인

각 화면과 네트워크 레이어에서 인증 정보를 직접 다루면 헤더 주입, 재발급 타이밍, 재시도 정책, 로그아웃 처리가 분산될 수 있음.

해결

- 인증 관련 기능을 모듈화하고 Logout 및 토큰 만료 시 자동 재인증 플로우 추가
- AuthDataStore는 토큰·유저 정보 영속화, AuthManager는 메모리 캐시와 로그아웃 이벤트 담당
- AuthInterceptor는 Authorization 헤더 자동 추가, TokenAuthenticator는 401 응답 시 토큰 재발급 및 재시도 처리
- Refresh Token 만료와 최대 재시도 초과 상황은 LogoutReason으로 분기해 UI에 전달

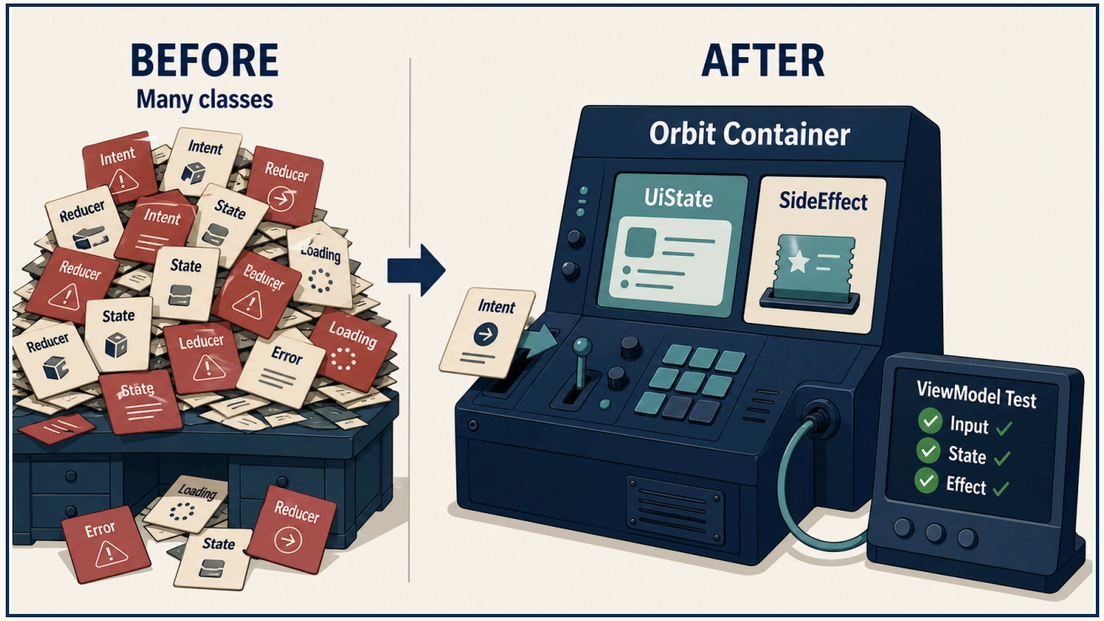
결과

- 인증 로직이 세 레이어로 분리되어 뷰/도메인 레이어에서 토큰 관리 코드 제거
- 토큰 만료·재발급·Refresh Token 만료·최대 재시도 초과 예외를 한 곳에서 제어
- LogoutReason으로 로그아웃 사유를 통합해 UI 메시지를 일관되게 노출

회고

AuthManager 중심 설계로 각 기능별 토큰 코드가 제거됐고, Coroutine + DataStore가 메모리 캐시와 영속화를 함께 해결해 안정적인 인증 시스템을 구축.

Orbit MVI 기반 상태 관리와 테스트 가능성 확보



ChatGPT로 제작한 이미지입니다.

문제

- 기존 MVI 구현 시 Intent/Reducer/State 클래스가 기하급수적으로 늘어나 코드베이스가 비대해짐
- 단순 입력 검증·로딩·에러 처리에도 보일러플레이트 코드가 증가하고 UI 테스트 범위가 불명확해짐

원인

화면별 Intent·Reducer 클래스를 개별 정의하면서 상태 관리 로직이 분산되고 UI에서 비즈니스 로직을 직접 처리함.

해결

- Orbit 기반 상태 관리 도입
- ContainerHost<UiState, SideEffect>와 container 하나로 State·SideEffect 통합 관리
- 화면 단위로 UiState, SideEffect를 분리 설계하여 책임 명확화 및 테스트 용이성 향상

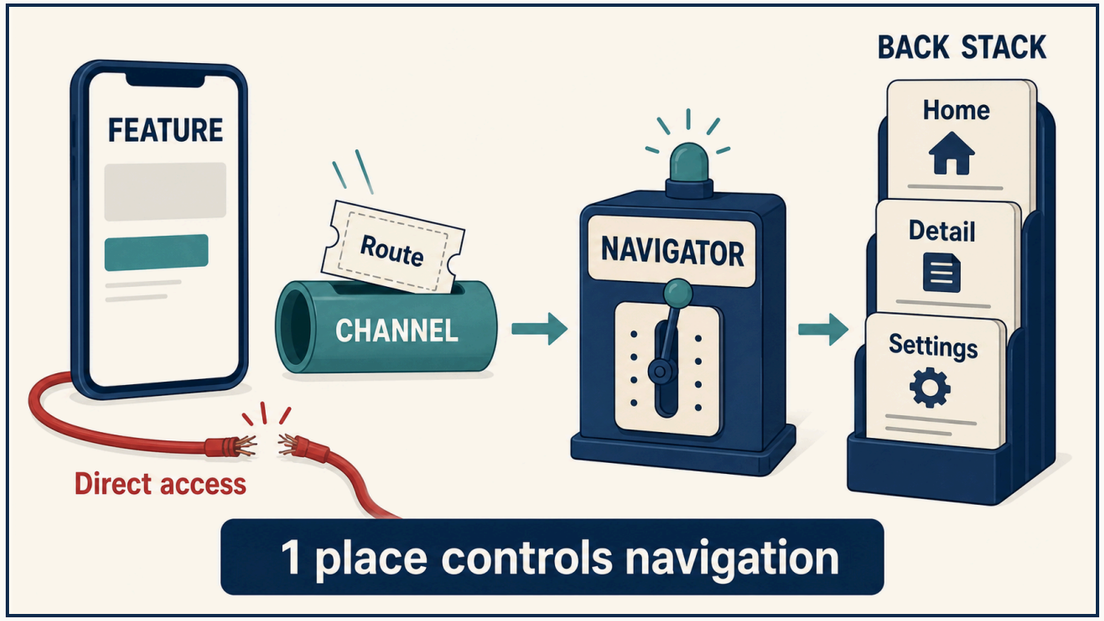
결과

- Intent·Reducer 클래스를 개별 정의하던 구조를 제거하고 Orbit 컨테이너 기반 단일 ViewModel 구조로 전환
- 입력 검증·로딩·에러 처리 로직을 테스트 가능한 ViewModel 계층에 집중
- State와 SideEffect를 분리해 화면 렌더링과 Toast/Navigation 같은 일회성 이벤트의 테스트 기준을 명확화
- UI 코드에서 비즈니스 로직을 분리하고 새로운 화면 추가 시 일관된 개발 플로우 확보

대체안 / 회고

sealed Intent + 단일 reducer, StateFlow + 확장 함수 조합도 가능했지만 검증된 컨테이너·SideEffect 처리를 활용하기 위해 Orbit을 선택. Intent·Reducer 클래스가 줄고 로직이 ViewModel로 집중되어 테스트 케이스 작성이 쉬워짐.

Channel 기반 Navigator로 화면-네비게이션 결합 분리



ChatGPT로 제작한 이미지입니다.

문제

- 각 Screen에서 NavBackStack에 직접 접근해 UI와 네비게이션 구현 상세 간 강한 결합 발생
- 화면 전환, BackStack 초기화, 뒤로 가기 처리 방식이 화면마다 달라 중복 코드와 일관성 저하 발생

원인

네비게이션 로직이 화면에 분산되어 전환 방식 변경 시 여러 화면을 수정해야 하고 문자열 기반 Route 오류 위험이 있었음.

해결

- Navigator 인터페이스와 NavigatorImpl을 별도 모듈에 정의하고 화면에서는 인터페이스만 의존
- Channel<InternalRoute>로 ViewModel에서 네비게이션 요청을 발행
- 네비게이션 전용 ViewModel 및 Composable에서 이벤트를 수신해 실제 NavBackStack 조작 수행

결과

- Screen에서 NavBackStack/Controller 직접 접근 제거, 화면과 네비게이션 구현 간 결합도 감소
- ViewModel은 navigator.navigate 호출만 수행하고 BackStack 조작은 전용 Composable이 담당
- 타입 기반 라우팅 모델 도입으로 문자열 기반 Route의 오타·런타임 오류 위험 감소
- 네비게이션 구현 변경 시 Feature 화면 수정 범위를 최소화

대체안 / 회고

NavController를 ViewModel에 주입하는 방법도 있었지만 UI와 네비게이션 결합이 남아 Channel 기반 패턴을 선택. 화면에서 네비게이션 코드가 제거되어 변경 시 한 곳만 수정하면 되는 구조가 됨.

Glim 성과 및 회고

성과 및 결과

- SSAFY 13th 구미캠퍼스 공통 프로젝트 우수상 수상
- AuthDataStore, AuthManager, AuthInterceptor, TokenAuthenticator 기반 인증 구조 안정적 구현
- Compose, Orbit MVI 기반 프로젝트 적용 및 화면 상태·SideEffect 처리 흐름 정리
- **ViewModel Unit Test**를 진행하여 안정적인 서비스 구축

회고

- Glim에서는 서로 다른 경험을 가진 팀원들이 모였기 때문에 처음부터 개발 방식이 완전히 맞지는 않았습니다.
- 코드 리뷰를 통해 각자의 구현 의도를 설명하고 조율하면서, 리뷰가 단순한 승인 절차가 아니라 팀의 기준을 맞추는 과정이라는 것을 체감했습니다.
- 로그인과 마이페이지처럼 사용자가 처음 마주하고 반복해서 쓰는 흐름을 맡으며, 인증/세션 로직은 화면마다 흩어지지 않고 역할별로 분리되어야 한다는 점을 배웠습니다.
- ViewModel 테스트를 많이 작성하면서 테스트 가능한 구조를 먼저 잡아야 기능이 늘어날 때 화면 상태와 예외 흐름을 안정적으로 검증할 수 있다는 점을 체감했습니다.